The first acceptance criteria, as well as the supporting unit tests, should now pass. Once you've verified this and done any necessary refactoring, you're ready to move on to the next acceptance criterion.

The process continues until you've written and implemented a set of low-level specifications for all of the classes involved in the high-level acceptance criteria. At this point, the acceptance criteria should pass.

10.2.8 How did BDD help?

Working this way gives you much finer control over what you're building and more confidence that the code actually works! Although this was a very simple requirement, the process we walked through did illustrate a number of important points.

THE ACCEPTANCE CRITERIA HELP YOU FOCUS ON VALUE

Starting with an acceptance criterion helps you remember what business value you're trying to deliver and provides a clear and convenient indicator so you know when you're done.

SMALL FEEDBACK LOOPS KEEP YOU ON TRACK

Working in very small feedback loops makes a lot of sense. The more code you write without the support of unit tests to back you up, the more chances you have of getting bogged down. The more code you write without a unit test to verify it, the more likely you'll write code that's not covered by a unit test, which both increases the risk of regressions and makes regression issues harder to isolate and fix when they do occur. For an experienced BDD or TDD practitioner, writing the unit tests first is a little like thinking aloud about the design.

THE UNIT TESTS ACT AS LOW-LEVEL SPECIFICATIONS AND DOCUMENTATION

In executable specifications like the previous ones, you're still describing behavior, but your target audience is different. The business stakeholders won't care about how a Frequent Flyer domain object is created, what its default values are, and so forth. But the developers who have to work with and maintain the code will. The unit tests you write are effectively technical specifications and documentation for other developers.

We've just walked through the process of taking a simple acceptance criterion and using it to produce a piece of working, verified production code. But this was a very simple case, and you could almost go directly from the step definitions to the production code. In the next section, we'll look at how to work with more involved acceptance criteria, where you'll need to use unit tests more actively to design and implement the production code.

10.3 Exploring low-level requirements, discovering design, and implementing more complex functionality

The acceptance criterion you implemented in the previous section was a very simple one, and it didn't require much in the way of unit tests or underlying code complexity. But this is usually not the case. In real-world applications, many acceptance criteria